

Facial Emotion Recognition Project Report

Project Summary

This project classifies cropped facial images into seven emotion categories using a custom convolutional neural network (CNN) and a MobileNetV2 transfer-learning model. The final saved notebook outputs show the custom CNN performed better on the held-out test split: **51.69% accuracy**, compared with **43.43%** for the cached-feature MobileNetV2 run. The CNN checkpoint, `best_cnn_model.keras`, is used by the Streamlit application in `app.py`.

The project also surfaced practical issues that affected iteration: slow sequential reads from Google Drive, multi-hour CNN training on CPU and severe class imbalance. The notebook and application were adjusted to address data-loading and deployment friction. Model performance is moderate, especially for the minority `disgust` class.

Project Scope and Data

The project does not collect new images from people or a camera. It reads an existing, folder-organized facial-expression dataset stored on mounted Google Drive at:

```
/content/drive/MyDrive/DeepFacialEmotionRecognition/FaceEmotionRecognitionDataset/images/train
```

The training directory contains **28,822 RGB images**, each with a resolution of **48 x 48 pixels**, organized into seven class folders. The class counts recorded in the notebook are:

Emotion	Images
Angry	3,993
Disgust	436
Fear	4,103
Happy	7,164
Neutral	4,982
Sad	4,939
Surprise	3,205
Total	28,822

The project reads images with OpenCV, converts them from BGR to RGB, and derives each label from its containing folder name. Labels are encoded with `LabelEncoder` and one-hot encoded for seven-way categorical cross-entropy training. The notebook reported no repeated filenames within class directories. Its main image loader raises an error if a file cannot be decoded.

The dataset was split using stratified sampling: 20% was held out for testing, then 20% of the remaining data was used for validation. This corresponds to approximately 64% training, 16% validation, and 20% test data overall. A fixed split seed (`random_state=42`) is set for both splits. CNN image values are converted to `float32` and normalized to `[0, 1]`.

For CNN training, `ImageDataGenerator` applies rotations up to 15 degrees, horizontal and vertical shifts up to 10%, zoom up to 10%, and horizontal flips. The minority `disgust` class is substantially underrepresented. No face detection or face cropping is performed by the application; uploaded images should already be close-up face crops similar to the training images.

Model Development

Custom CNN

The custom CNN has four convolutional blocks with 64, 128, 256, and 512 filters. Each block uses a 3 x 3 convolution, batch normalization, max pooling, and dropout. The dense classifier has 256 and 128 units with dropout, followed by a seven-unit softmax output. The current dropout rates are 0.15, 0.15, 0.20, and 0.20 in the convolutional blocks, then 0.35 and 0.25 in the dense layers.

The CNN is compiled with Adam and categorical cross-entropy. It trains on augmented batches of 64 images for up to 30 epochs. Early stopping monitors validation accuracy and restores the best weights; a model checkpoint saves the best validation-accuracy model to `best_cnn_model.keras`. `ReduceLROnPlateau` lowers the learning rate when validation loss plateaus.

MobileNetV2

The second experiment uses ImageNet-pretrained MobileNetV2 without its classification top. The backbone is frozen and accepts the dataset's native 48 x 48 resolution. Inputs use MobileNetV2 preprocessing, mapping pixel values into the range expected by its pretrained weights. Global average pooling produces 1,280-dimensional features, followed by a 256-unit dense layer, dropout, and a seven-class output layer.

Because the backbone is frozen, its pooled features are computed once for the training, validation, and test splits. The smaller dense head then trains on those cached features. This substantially reduces repeated CPU work compared with recomputing the backbone on every epoch.

Evaluation and Findings

The following are the saved results for the recent 48 x 48 CNN and cached-feature MobileNetV2 runs in the notebook:

Model	Best validation accuracy	Test accuracy	Test macro F1	Test weighted F1
Custom CNN	51.61%	51.69%	0.40	0.49
MobileNetV2, frozen cached features	43.84%	43.43%	Not recorded in the cited saved output	Not recorded in the cited saved output

The CNN's saved classification report shows uneven class performance:

Emotion	Precision	Recall	F1
Angry	0.53	0.09	0.15
Disgust	0.00	0.00	0.00
Fear	0.32	0.20	0.25
Happy	0.81	0.83	0.82
Neutral	0.43	0.60	0.50
Sad	0.32	0.54	0.40
Surprise	0.72	0.67	0.69

The CNN performs best on happy and surprise, with moderate neutral and sad performance. Angry and fear recall are low, and the model does not correctly identify `disgust` in the saved test report. Overall accuracy therefore overstates how evenly the model recognizes all seven emotions. The macro F1 score (0.40) is important context alongside accuracy.

The MobileNetV2 head's final training accuracy was about 64%, while its best validation accuracy was about 43.84%. This train/validation gap indicates overfitting of the classifier head or a mismatch between ImageNet features and this low-resolution emotion task. In this experiment the custom CNN had stronger validation and test results, so it is the selected model for the Streamlit prototype.

The model comparison used the same saved test split, but choosing a model after inspecting test scores makes the test set part of model selection. For a formal final estimate, choose the model using validation metrics first, then evaluate the selected model on an untouched test set only once.

Development Challenges and Resolutions

Slow image loading from Drive

The initial unreadable-image scan decoded every image sequentially, and the later data loader decoded the same files again. With 28,822 files on mounted Drive, this caused long waits. The loader was changed to use a bounded thread pool and report progress after each batch of 512 images. This avoids repeating the full read and makes the loading stage observable.

Long CPU training time

The saved CNN log shows approximately 550 to 579 seconds per epoch for 289 batches, with all 30 configured epochs running. That is roughly 4.6 hours for the CNN fit. The MobileNetV2 experiment addresses repeated backbone work by caching features once, using native 48 x 48 inputs, and training only the compact classifier head across epochs. On native Windows, the installed TensorFlow build runs on CPU; TensorFlow 2.21 reports that native Windows GPU support is unavailable.

CNN initially collapsed to one class

Initial runs stayed near 25% accuracy and predicted happy for almost all samples, approximately matching the share of the largest class. The project reduced CNN dropout, stratified the validation split, removed aggressive class weighting from the accuracy-focused run, and aligned early stopping with the validation-accuracy checkpoint. The later saved run learned multiple classes and reached 51.69% test accuracy, although `disgust` recall remains zero.

Streamlit Integration

`app.py` loads `best_cnn_model.keras` from the same folder as the application and caches the model between Streamlit reruns. For each uploaded image it:

1. Converts it to RGB.
2. Resizes it to 48 x 48 pixels.
3. Converts it to `float32`, divides by 255, and adds a batch dimension.
4. Predicts one of the seven classes and displays the top class, its raw softmax score, and per-class scores.

The class order is `angry`, `disgust`, `fear`, `happy`, `neutral`, `sad`, `surprise`, matching the notebook's `LabelEncoder` ordering and the seven output units. The displayed softmax score is not calibrated confidence. The app accepts cropped face images; it does not locate faces inside a larger scene.

Install the app dependencies into the environment and launch from the project folder:

```
C:\ferenv\Scripts\python.exe -m pip install tensorflow streamlit pillow
C:\ferenv\Scripts\python.exe -m streamlit run app.py
```

Reproducibility and Limitations

- The image directory is mounted from Google Drive in the notebook; the dataset itself is not included in the model artifact.
- The notebook relies on `os.listdir` for class/file enumeration. The split seed is fixed, but file ordering and model initialization may vary between environments unless the ordering and TensorFlow/NumPy random seeds are also fixed.
- The classes are imbalanced, especially `disgust` (436 images versus 7,164 happy images). This strongly affects minority-class recall.
- The current app performs classification on the whole uploaded image after resizing. It does not run a face detector or crop a face automatically.

- Emotion labels are subjective and model outputs should not be interpreted as a reliable assessment of a person's internal emotional state.
- Metrics in this report are transcribed from the notebook's saved outputs. Different hardware, package versions, random initialization, or training runs may produce different results.

Conclusion

The custom CNN is the best-performing model among the two current experiments and is integrated into the Streamlit app through `best_cnn_model.keras`. The project now has an end-to-end path from folder-based image data, through stratified splits and model evaluation, to interactive single-image inference. The strongest next improvement is class-wise: address the weak angry, fear, and especially disgust recall while monitoring macro F1 and validation performance. Any new model should be selected with validation data before its one-time final test evaluation.